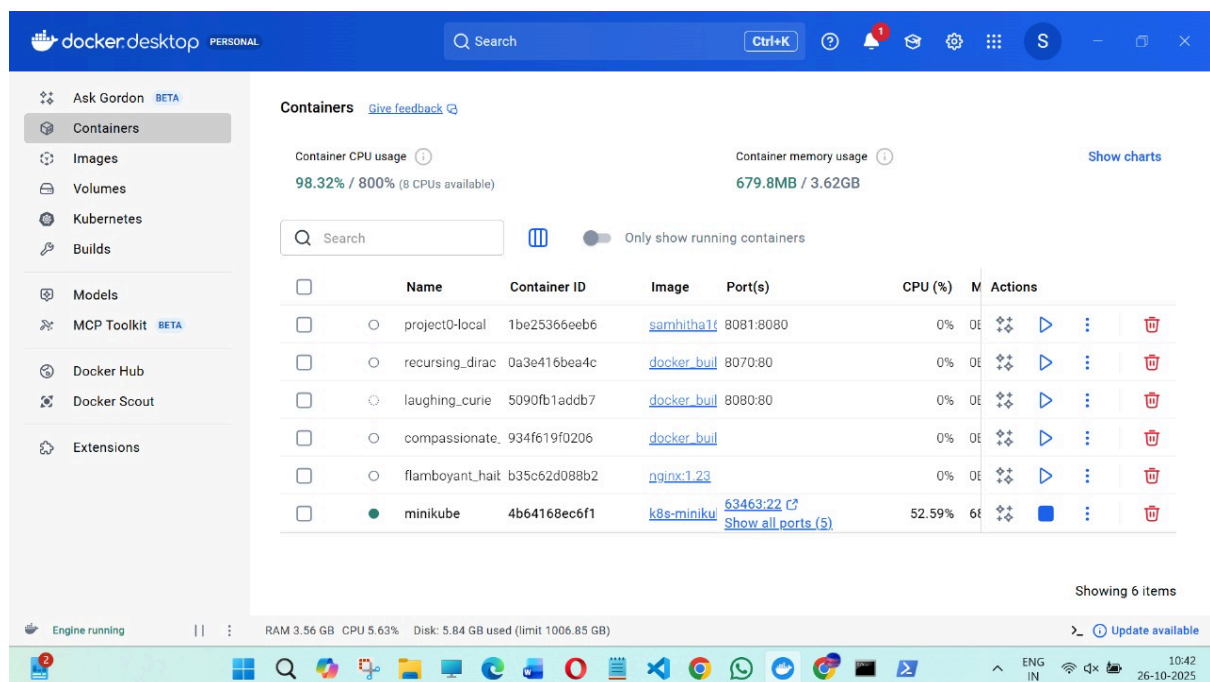


Kubernetes:

Kubernetes is an open-source container orchestration platform that automates the deployment, scaling, and management of containerized applications. It allows multiple containers to be organized into logical units, making complex deployments easier to manage and monitor. Kubernetes ensures that applications run reliably by automatically handling container scheduling, load balancing, and self-healing in case of failures.

In this project, Kubernetes was implemented to manage and orchestrate Docker containers created in the previous stage of the DevOps pipeline. By using Kubernetes, the application deployment became more scalable, fault-tolerant, and efficient. The cluster automatically monitors container health and replaces failed instances, ensuring continuous application availability. This reduces manual intervention and simplifies the overall maintenance process.

Through Kubernetes, we also achieved an improved CI/CD workflow. The application packaged in Docker images was deployed into pods managed by Kubernetes, allowing for easy updates and rollbacks. It provides flexibility for scaling workloads up or down based on demand, maintaining optimal resource usage. This automation aligns with DevOps principles, promoting faster, more reliable software delivery across different environments.



Deploying an Application on Kubernetes

1. Create a Deployment File

To deploy an application in Kubernetes, a Deployment YAML file is created. This file defines how many replicas (pods) to run, what image to use, and how the containers should behave.

Below is an example deployment configuration:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-kube-app
spec:
  replicas: 2
  selector:
    matchLabels:
      app: my-kube-app
  template:
    metadata:
      labels:
        app: my-kube-app
    spec:
      containers:
        - name: my-kube-container
          image: my-docker-app:latest
          ports:
            - containerPort: 80
```

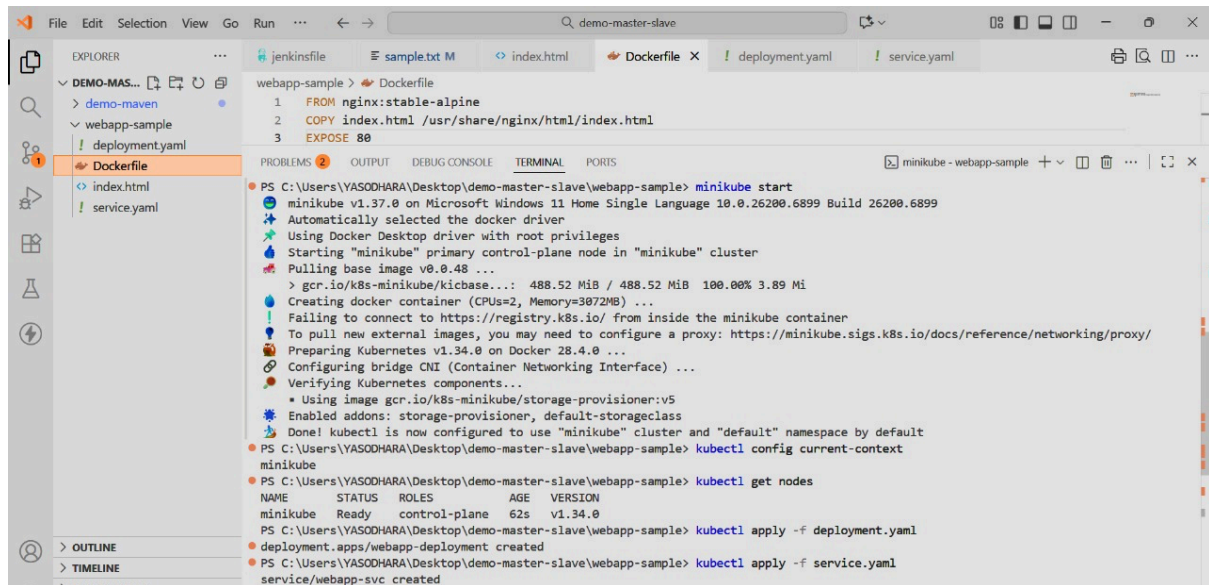
This configuration ensures that two instances of the application container are created and managed by Kubernetes. If one pod fails, another is automatically started to maintain availability.

2. Apply the Deployment

Once the deployment file is created, it is applied using the following command:

```
kubectl apply -f deployment.yaml
```

This command instructs Kubernetes to create the defined deployment and manage the pods based on the configuration provided. Kubernetes will automatically pull the specified Docker image, create the pods, and start the containers inside them.



3. Verify the Deployment

After applying the deployment, verify that the pods are running successfully using the command:

```
kubectl get pods
```

This command lists all pods currently managed by Kubernetes. Each pod will have a unique name and a status of **Running** if the deployment was successful.

Exposing the Application using a Service

1. Create a Service File

After deploying the application, the next step is to expose it so it can be accessed from outside the cluster. To achieve this, a Service YAML file is created. The service acts as a network gateway that routes external traffic to the pods running inside the cluster.

Below is an example of a service configuration:

```
apiVersion: v1
kind: Service
metadata:
  name: my-kube-service
spec:
  type: NodePort
  selector:
```

```
app: my-kube-app
ports:
- protocol: TCP
  port: 80
  targetPort: 80
  nodePort: 30008
```

Here:

- type: NodePort exposes the application on a port accessible from the host machine.
- selector ensures traffic is routed to the correct pods.
- nodePort: 30008 specifies the port through which the application can be accessed externally.

2. Apply the Service

Once the Service YAML file is ready, apply it using the following command:

```
kubectl apply -f service.yaml
```

This command creates a new service resource and binds it to the running pods defined in the deployment. Kubernetes assigns a unique cluster IP and maps it to the specified NodePort for external access.

3. Verify the Service

To confirm that the service is running, use the command:

```
kubectl get services
```

This lists all active services, along with their cluster IPs, external ports, and types. The my-kube-service should appear in the list with the NodePort 30008 mapped to port 80, indicating that the application is now exposed successfully.

Conclusion

In this part of the project, Kubernetes was implemented to manage and orchestrate Docker containers efficiently. Using Kubernetes, the deployment process became automated, scalable, and fault-tolerant. By defining deployments and services through YAML configuration files, the application could be easily deployed, updated, and maintained without manual intervention.

Kubernetes ensured that the application containers were always running as intended. If any container failed, Kubernetes automatically replaced it, maintaining high availability. The use of services enabled reliable communication and external access to the application, ensuring consistent performance for end users. This approach simplified operational management and eliminated the need for manual scaling or recovery.

Through this implementation, we achieved improved resource utilization, streamlined deployment, and enhanced application stability. Kubernetes provided the foundation for an enterprise-ready deployment strategy by integrating seamlessly with Docker, completing the DevOps workflow established in this project. Together, Docker and Kubernetes made the pipeline fully automated, portable, and production-ready.